

Software Engineer (Flights OTA) Coding Exercise v1.0

Overview

This coding exercise involves designing and implementing an API-only application that returns the closest car parks to a user, together with each parking lot's availability.

The choices of framework, or tools are up to you but please do it in **Java**. You should also include the use of a **relational database**, such as MySQL or PostgreSQL (if there is a good reason not to use a relational database, tell us!)

You are strongly encouraged to use AI tools (e.g., Cursor, Windsurf, CoPilot, Claude Code, Gemini CLI) to complete or assist with this assignment. Demonstrate your experience by sharing the prompts or techniques you used.

Data sources

- **Car park information**

Dataset: https://beta.data.gov.sg/datasets/d_23f946fa557947f93a8043bbef41dd09/view

This dataset provides detailed information about each car park. This can be treated as static and is to be loaded with a task from the CSV file provided in the link above. You can check it into the repository and do not have to automate the downloading and updating of this data.

Hint: The coordinates provided are in a SVY21 format. You may have to do some conversion to a more widely used format. An option is to use a coordinate converter 3414(SVY21) to 4326(WGS84) from this list of APIs (<https://www.onemap.gov.sg/apidocs/apidocs>) when importing or a similarly implemented library.

- **Car park availability**

Dataset: https://beta.data.gov.sg/datasets/d_ca933a644e55d34fe21f28b8052fac63/view

API Endpoint: <https://api.data.gov.sg/v1/transport/carpark-availability>

The API endpoint in that link provides live updates on the parking lot availability for the car parks. Create a task to call the API and save this data in the database. Scheduling the task is out of the scope of this project, but if manually run, the same task should be able to update existing data with newer data from the API if run periodically.

Business Requirements

- The endpoint should take the URL parameters `latitude` and `longitude`, and return a JSON array of car parks sorted by distance ascending with the total and available parking lots.
- Requests without required parameters `latitude` and `longitude` should return HTTP status code `400`.
- Pagination should be implemented with `page` and `per_page` to iterate through a subset of results. Only car parks with available parking lots should be returned.
- Input validation should be implemented where you see fit.

Expected output

Request

```
GET /carparks/nearest?latitude=1.37326&longitude=103.897&page=1&per_page=3
```

Response

```
[
  {
    "address": "BLK 401-413, 460-463 HOUGANG AVENUE 10",
    "latitude": 1.37429,
    "longitude": 103.896,
    "total_lots": 693,
    "available_lots": 182
  },
  {
    "address": "BLK 351-357 HOUGANG AVENUE 7",
    "latitude": 1.37234,
    "longitude": 103.899,
    "total_lots": 249,
    "available_lots": 143
  },
  {
    "address": "BLK 364 / 365 UPP SERANGOON RD",
    "latitude": 1.37011,
    "longitude": 103.897,
    "total_lots": 471,
    "available_lots": 324
  }
]
```

Some rules

- Use version control (preferably Git, but any will do), and use it as you would for your regular work.
- Package your development environment with something like Docker Compose.
- Your application should be in a state that is **runnable** by our reviewers.
- Include documentation in a top level README.md, explaining the approach, instructions on how to run your application, and any other details as you see fit.
- Include your prompts with AI tools, so we understand how you use them.
- We use automation tools to filter the submission before it reaches our reviewers. So please make sure you follow the API specs correctly to avoid unfortunate rejection.

For bonus points

- We love clean, readable code that comes with tests.
- Keep in mind scalability, performance, and maintainability concerns.
- We love to hear other people's thought process and the tradeoffs they consider, so write those down.

Guidelines

We understand that your time is precious, so please aim to spend at most 8 hours on this exercise in total. As a guideline, consider spending between 3-8 hours and then calling a hard stop.

If you still have ideas to improve the solution but stopped because of the time limit, please feel free to share them in README.md

Please try to submit within 1 week, but if you need more time due to personal or work commitments, just drop your recruitment coordinator a note.

Submission

Please host the code for your submission on **GitHub** as a **private** repository, and then invite:

- [lj-wego](#)
- [viet-wego](#)
- [tintran-wego](#)
- [redha-wego](#)

Drop LJ (liangjun@wego.com) an email when you're done.